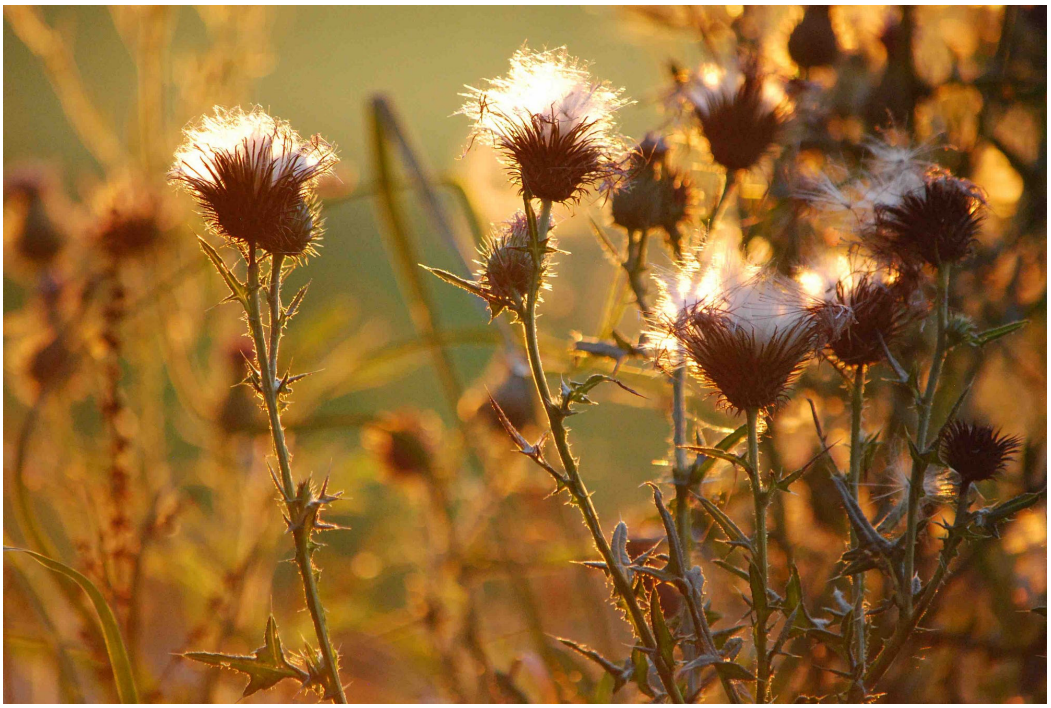
Preprocessing for Deep Learning: From covariance matrix to image whitening

The goal of this post/notebook is to go from the basics of data preprocessing to modern techniques used in deep learning. My point is that we can use code (Python/Numpy etc.) to better understand abstract mathematical notions!

By **Hadrien Jean**, Machine Learning Scientist on October 10, 2018 in **Data Preprocessing**, **Deep Learning**, **Image Processing**, **Mathematics**

comments



The goal of this post is to go from the basics of data preprocessing to modern techniques used in deep learning. My point is that we can use code (such as Python/NumPy) to better understand abstract mathematical notions. Thinking by coding! ????

We will start with basic but very useful concepts in data science and machine learning/deep learning, like variance and covariance matrices. We will go further to some preprocessing techniques used to feed images into neural networks. We will try to get more concrete insights using code to actually see what each equation is doing.

10 GitHub Repositories to Master Vibe Coding

Building a Simple Data Quality DSL in Python

7 ChatGPT Tricks to Automate Your Data Tasks

Context Engineering is the New Prompt Engineering

5 Practical Docker Configurations

Top 5 Small AI Coding Models That You Can Run Locally

7 AI Tools I Can't Live Without as a Professional Data Scientist

Staying Ahead of AI in Your Career

5 Critical Feature Engineering Mistakes That Kill Machine Learning Projects

5 Excel AI Lessons I Learned the Hard Way

Preprocessing refers to all the transformations on the raw data before it is fed to the machine learning or deep learning algorithm. For instance, training a convolutional neural network on raw images will probably lead to bad classification performances (Pal & Sudeep, 2016). The preprocessing is also important to speed up training (for instance, centering and scaling techniques, see Lecun et al., 2012; see 4.3).

Here is the syllabus of this tutorial:

1. Background: In the first part, we will get some reminders about variance and covariance. We will see how to generate and plot fake data to get a better understanding of these concepts.

2. Preprocessing: In the second part we will see the basics of some preprocessing techniques that can be applied to any kind of data—**mean normalization**, **standardization**, and **whitening**.

3. Whitening images: In the third part, we will use the tools and concepts gained in **1.** and **2.** to do a special kind of whitening called **Zero Component Analysis** (ZCA). It can be used to preprocess images for deep learning. This part will be very practical and fun ☃!

Feel free to fork the notebook associated with this post! For instance, check the shapes of the matrices each time you have a doubt.

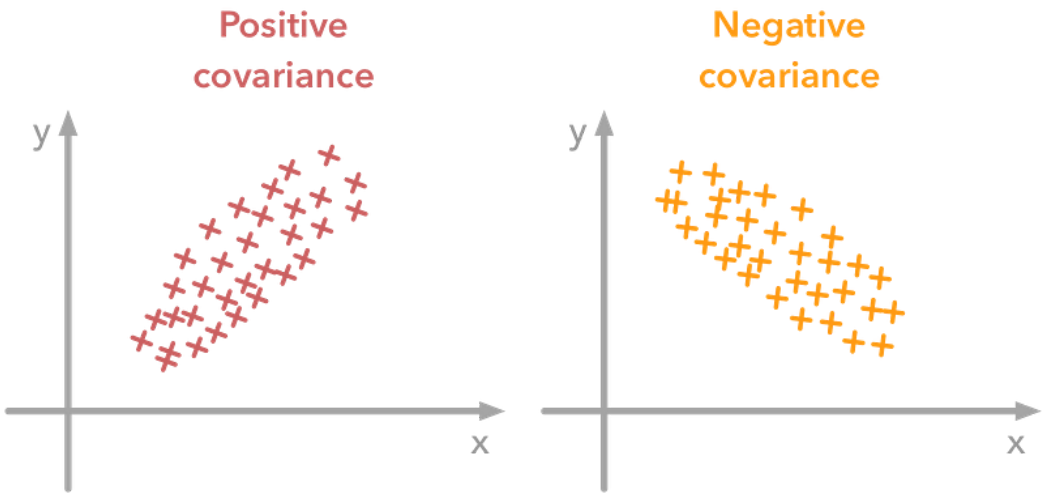
1. Background

A. Variance and covariance

The variance of a variable describes how much the values are spread. The covariance is a measure that tells the amount of dependency between two variables.

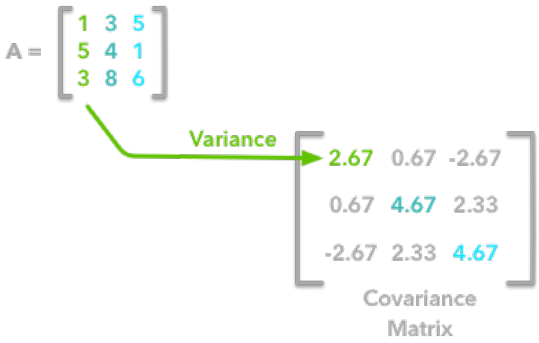
A positive covariance means that the values of the first variable are large when values of the second variables are also large. A negative covariance means the opposite: large values from one variable are associated with small values of the other.

The covariance value depends on the scale of the variable so it is hard to analyze it. It is possible to use the correlation coefficient that is easier to interpret. The correlation coefficient is just the normalized covariance.



A positive covariance means that large values of one variable are associated with big values from the other (left). A negative covariance means that large values of one variable are associated with small values of the other one (right).

The covariance matrix is a matrix that summarises the variances and covariances of a set of vectors and it can tell a lot of things about your variables. The diagonal corresponds to the variance of each vector:



A matrix ***A*** and its matrix of covariance. The diagonal corresponds to the variance of each column vector.

Let's just check with the formula of the variance:

$$V(\boldsymbol{X}) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

with **n** the length of the vector, and **x̄** the mean of the vector. For instance, the variance of the first column vector of **A** is:

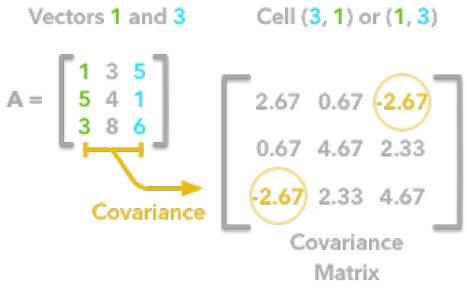
$$V(\boldsymbol{A}_{:,1}) = \frac{(1-3)^2 + (5-3)^2 + (3-3)^2}{3} = 2.67$$

This is the first cell of our covariance matrix. The second element on the diagonal

corresponds of the variance of the second column vector from **A** and so on.

Note: the vectors extracted from the matrix **A** correspond to the columns of **A**.

The other cells correspond to the covariance between two column vectors from ***A***. For instance, the covariance between the first and the third column is located in the covariance matrix as the column 1 and the row 3 (or the column 3 and the row 1).



The position in the covariance matrix. Column corresponds to the first variable and row to the second (or the opposite). The covariance between the first and the third column vector of ***A*** is the element in column 1 and row 3 (or the opposite = same value).

Let's check that the covariance between the first and the third column vector of **A** is equal to -2.67. The formula of the covariance between two variables ***X***and **Y** is:

$$cov(\boldsymbol{X}, \boldsymbol{Y}) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})$$

The variables **X** and **Y** are the first and the third column vectors in the last example. Let's split this formula to be sure that it is crystal clear:

$$(x_1 - \bar{x})$$

1. The sum symbol (**Σ**) means that we will iterate on the elements of the vectors. We will start with the first element (**i=1**) and calculate the first element of **X** minus the mean of the vector **X**.

$$(x_1 - \bar{x})(y_1 - \bar{y})$$

2. Multiply the result with the first element of **Y** minus the mean of the vector ***Y***.

$$\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})$$

3. Reiterate the process for each element of the vectors and calculate the sum of all results.

$$\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})$$

4. Divide by the number of elements in the vector.

Example 1.

Let's start with the matrix **A**:

$$\boldsymbol{A} = \begin{bmatrix} 1 & 3 & 5 \\ 5 & 4 & 1 \\ 3 & 8 & 6 \end{bmatrix}$$

We will calculate the covariance between the first and the third column vectors:

$$\boldsymbol{X} = \begin{bmatrix} 1 \\ 5 \\ 3 \end{bmatrix}$$

and

$$\boldsymbol{Y} = \begin{bmatrix} 5 \\ 1 \\ 6 \end{bmatrix}$$

x̄=3, **ȳ=4**, and **n=3** so we have:

$$cov(X,Y) = \frac{(1-3)(5-4) + (5-3)(1-4) + (3-3)(6-4)}{3} = \frac{-8}{3} = -2.67$$

Ok, great! That's the value of the covariance matrix.

Now the easy way. With NumPy, the covariance matrix can be calculated with the

function `np.cov`.

It is worth noting that if you want NumPy to use the columns as vectors, the parameter `rowvar=False` has to be used. Also, `bias=True` divides by **n** and not by **n-1**.

Let's create the array first:

```
array([[1, 3, 5],
       [5, 4, 1],
       [3, 8, 6]])
```

Now we will calculate the covariance with the NumPy function:

```
array([[ 2.66666667, 0.66666667, -2.66666667],
       [ 0.66666667, 4.66666667, 2.33333333],
       [-2.66666667, 2.33333333, 4.66666667]])
```

Looks good!

Finding the covariance matrix with the dot product

There is another way to compute the covariance matrix of **A**. You can center **A** around 0. The mean of the vector is subtracted from each element of the vector to have a vector with mean equal to 0. It is multiplied with its own transpose, and divided by the number of observations.

Let's start with an implementation and then we'll try to understand the link with the previous equation:

Let's test it on our matrix **A**:

```
array([[ 2.66666667, 0.66666667, -2.66666667],
       [ 0.66666667, 4.66666667, 2.33333333],
       [-2.66666667, 2.33333333, 4.66666667]])
```

We end up with the same result as before.

The explanation is simple. The dot product between two vectors can be expressed:

$$\boldsymbol{X}^{\mathrm{T}}\boldsymbol{Y} = \sum_{i=1}^{n}(x_i)(y_i)$$

That's right, it is the sum of the products of each element of the vectors:

$$\begin{matrix} & x & & y \\ \begin{bmatrix} 1 & 7 & 2 \end{bmatrix} & & \begin{bmatrix} 3 \\ 5 \\ 2 \end{bmatrix} & = 1 \times 3 + 7 \times 5 + 2 \times 2 \end{matrix}$$

$$= x_1 y_1 + x_2 y_2 + x_2 y_2$$

$$= \sum(x_i y_i)$$

The dot product corresponds to the sum of the products of each element of the vectors.

If **n** is the number of elements in our vectors and that we divide by **n**:

$$\frac{1}{n}\boldsymbol{X}^{\mathrm{T}}\boldsymbol{Y} = \frac{1}{n}\sum_{i=1}^{n}(x_i)(y_i)$$

You can note that this is not too far from the formula of the covariance we have seen earlier:

The only difference is that, in the covariance formula, we subtract the mean of a vector from each of its elements. This is why we need to center the data before doing the dot product.

Now, if we have a matrix **A**, the dot product between **A** and its transpose will give you a new matrix:

$$\begin{bmatrix} - & x & - \\ - & y & - \end{bmatrix} \begin{bmatrix} | & | \\ x & y \\ | & | \end{bmatrix} = \begin{bmatrix} \sum x^2 & \sum yx \\ \sum xy & \sum y^2 \end{bmatrix}$$

If you start with a zero-centered matrix, the dot product between this matrix and its transpose will give you the variance of each vector and covariance between them, that is to say the covariance matrix.

This is the covariance matrix!

B. Visualize data and covariance matrices

In order to get more insights about the covariance matrix and how it can be useful, we will

create a function to visualize it along with 2D data. You will be able to see the link between the covariance matrix and the data.

This function will calculate the covariance matrix as we have seen above. It will create two subplots—one for the covariance matrix and one for the data. The `heatmap()` function from Seaborn is used to create gradients of colour—small values will be coloured in light green and large values in dark blue. We chose one of our palette colours, but you may prefer other colours. The data is represented as a scatterplot.

C. Simulating data

Uncorrelated data

Now that we have the plot function, we will generate some random data to visualize what the covariance matrix can tell us. We will start with some data drawn from a normal distribution with the NumPy function `np.random.normal()`.

Drawing sample from a normal distribution with NumPy.

This function needs the mean, the standard deviation and the number of observations of the distribution as input. We will create two random variables of 300 observations with a standard deviation of 1. The first will have a mean of 1 and the second a mean of 2. If we randomly draw two sets of 300 observations from a normal distribution, both vectors will be uncorrelated.

Note 1: We transpose the data with `.T` because the original shape is `(2, 300)` and we want the number of observations as rows (so with shape `(300, 2)`).

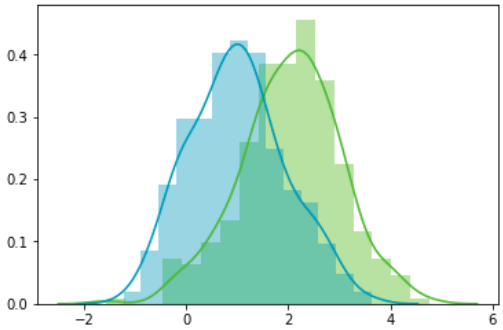
Note 2: We use `np.random.seed` function for reproducibility. The same random number will be used the next time we run the cell.

Let's check how the data looks like:

```
array([[ 2.47143516, 1.52704645],
       [ 0.80902431, 1.7111124 ],
       [ 3.43270697, 0.78245452],
       [ 1.6873481 , 3.63779121],
       [ 1.27941127, -0.74213763],
       [ 2.88716294, 0.90556519],
       [ 2.85958841, 2.43118375],
       [ 1.3634765 , 1.59275845],
       [ 2.01569637, 1.1702969 ],
       [-0.24268495, -0.75170595]])
```

Nice, we have two column vectors.

Now, we can check that the distributions are normal:

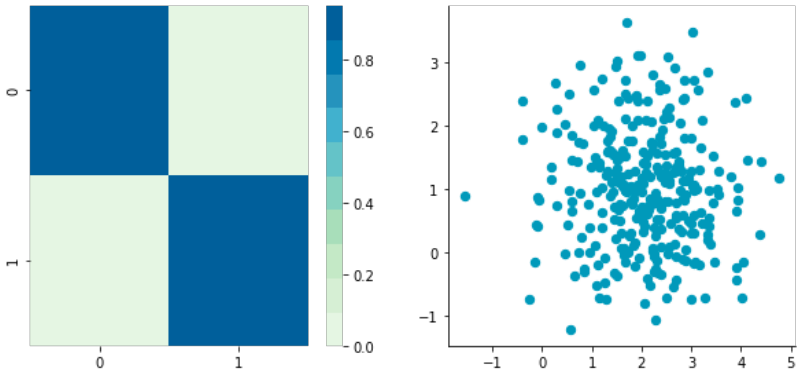


Looks good!

We can see that the distributions have equivalent standard deviations but different means (1 and 2). So that's exactly what we have asked for.

Now we can plot our dataset and its covariance matrix with our function:

```
Covariance matrix:
[[ 0.95171641 −0.0447816 ]
 [−0.0447816 0.87959853]]
```



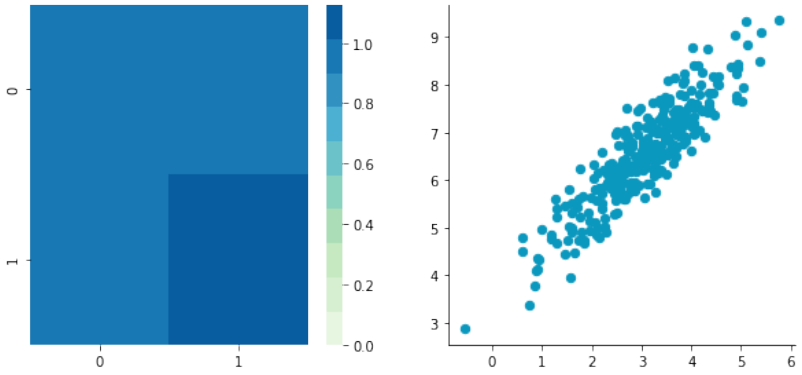
We can see on the scatterplot that the two dimensions are uncorrelated. Note that we have one dimension with a mean of 1 (y-axis) and the other with the mean of 2 (x-axis).

Also, the covariance matrix shows that the variance of each variable is very large (around 1) and the covariance of columns 1 and 2 is very small (around 0). Since we ensured that the two vectors are independent this is coherent. The opposite is not necessarily true: a covariance of 0 doesn't guarantee independence (see here).

Correlated data

Now, let's construct dependent data by specifying one column from the other one.

```
Covariance matrix:
[[ 0.95171641 0.92932561]
 [ 0.92932561 1.12683445]]
```



The correlation between the two dimensions is visible on the scatter plot. We can see that a line could be drawn and used to predict **y** from **x** and vice versa. The covariance matrix is not diagonal (there are non-zero cells outside of the diagonal). That means that the covariance between dimensions is non-zero.

That's great! We now have all the tools to see different preprocessing techniques.

More On This Topic

- Vector and Matrix Norms with NumPy Linalg Norm
- How to Perform Matrix Operations with NumPy
- 7 Steps to Mastering Data Cleaning and Preprocessing Techniques
- Harnessing ChatGPT for Automated Data Cleaning and Preprocessing
- Learn Data Cleaning and Preprocessing for Data Science with This Free eBook
- Cleaning and Preprocessing Text Data in Pandas for NLP Tasks